

Why AI changed the game, even for people who aren't using AI

I'll be honest. A year ago I thought a lot of the "AI changes everything in security" talk was overhyped. It felt like every vendor was slapping AI onto their pitch deck. Then I watched a junior engineer at a meetup write, in about forty minutes, a passable API fuzzer using nothing but an LLM and a coffee. No security background. No exploit knowledge. Just curiosity and the ability to ask good questions.

That moment changed my mind. The barrier to entry for poking at APIs has collapsed. What used to require some skill and patience now requires the ability to type. Attackers don't need to be experts anymore. They need an idea and access to a model.

And here's the other side. AI agents are now legitimate callers of APIs. Tools like assistants and automations are out there in the wild, making API calls on behalf of real users. They sometimes make weird requests. They sometimes hallucinate parameters. They sometimes loop because they think they got the wrong answer the first time. Your API needs to be okay with this, but also not be easily tricked by it.

So you've got two trends pushing at the same time. The pool of people who can attack your API just got way bigger. And the pool of legitimate non-human callers also just got way bigger. Distinguishing between the two is the new game.

The five things that go wrong most often

After looking at way too many API breach post-mortems, I've started to notice the same handful of issues coming up over and over. The OWASP API Security Top 10 covers them formally if you want the official list, but here's the plain-English version of the ones I see most.

- *Broken object-level authorisation.* This is the one my friend got hit

by. Your endpoint takes an ID, returns the object, but doesn't check whether the caller actually has the right to see that object. Embarrassingly common.

- *Weak or missing authentication.* Tokens that don't expire. Tokens whose signatures aren't being verified. Basic auth with hard-coded credentials still in use because it works and nobody wants to touch it.
- *Excessive data exposure.* Your endpoint returns the full user object including the password hash and the internal admin notes, and the frontend just shows the name. The frontend hiding field is not the same as the backend not sending them.
- *Missing rate limits.* An endpoint that can be called ten thousand times in a minute by a single client is essentially an open door for scraping, brute force, and denial of service.
- *Outdated dependencies.* Half the breaches I read about start with a library that had a known vulnerability which nobody patched. Boring. Preventable. Still happens constantly.

If you fix nothing else this quarter, fix these five. I mean it.

Authentication and authorisation are not the same thing

This trips up so many developers that it's worth saying clearly. Authentication is who you are. Authorisation is what you're allowed to do.

Authentication is mostly a solved problem at this point. OAuth 2.0, OpenID Connect, JWTs signed with proper algorithms. The libraries exist. The patterns are well known. You can mess it up, but the path to doing it right is clear.

Authorisation is where most apps fall apart. And it's harder because it depends on your specific business logic. There's no library that knows whether user 47291 should be allowed to update invoice 88420. That depends on whether user 47291 owns the invoice, or works for the company that owns it, or has

been delegated permission, or any number of other things that are unique to your product.

Here's the simplest version of what an authorisation check on a per-object basis might look like in a node service:

```
app.get('/api/invoices/:id',
  authenticate, async (req, res) => {
    const invoice = await db.invoices.
      findById(req.params.id);
    if (!invoice) return res.status(404).
      send('Not found');

    if (invoice.ownerId !== req.user.id &&
      req.user.role !== 'admin') {
      return res.status(403).
        send('Forbidden');
    }
    res.json(invoice);
  });
```

It's three extra lines. Three lines that prevent your API from becoming the next inventory script story. Yet I see endpoints without this check probably more often than I see endpoints with it. The trick is to make this check impossible to forget. Either build a middleware that runs on every request and checks ownership, or push authorisation into a policy layer like Open Policy Agent so it's centralised and reviewable. Whatever you do, don't rely on every developer to remember every time.

Microservices made the problem harder, then easier, then harder again

When microservices first became the thing, a lot of teams ran into a weird security problem. Inside the cluster, services trusted each other completely. The reasoning was: we're all behind the same firewall, so what could go wrong? Well, quite a lot, especially once attackers got a foothold in any one service and could pivot freely from there.

The industry's answer to this has been mutual TLS and service meshes. Both ends of every internal